

Software Products (Ian Sommerville)

1. Introduction

1. Software products are generic software systems that provide useful functionality to many customers.
 2. Examples:
 - Large-scale systems – MS Excel, SAP.
 - Personal products – Evernote, Notion.
 - Mobile apps and games – Sudoku, Instagram.
 3. Software product engineering evolved from techniques used for custom (one-off) systems.
-

2. Types of Software Development

A. Custom (Project-based) Software Engineering

1. Begins with software requirements from an external client.
2. Contractor designs and implements the system to meet those requirements.
3. Requirements may change as business needs evolve.
4. Systems usually have long lifetimes (10+ years) and require maintenance and support.
5. Examples: government information systems, enterprise ERPs.

B. Product Software Engineering

1. Begins from a business opportunity rather than specific customer requests.
2. The company decides on features, timeline, and release schedule.
3. Goal is rapid delivery to capture and maintain market share.

4. Examples: mobile apps, software as a service (SaaS) tools.
-

3. Software Execution Models

1. **Stand-alone:** Entire software runs on the user's computer (e.g., MS Word).
 2. **Hybrid:** Some functionality runs locally; other features run on the developer's servers (e.g., Dropbox).
 3. **Software as a Service (SaaS):** Entire system runs on the developer's servers and is accessed through a web browser or mobile app (e.g., Google Docs).
-

4. Product Vision

1. A product vision is a short statement describing:
 1. What the product is.
 2. Who the target users are.
 3. Why customers should buy or use it.
2. Moore's Template:
 1. FOR (target customer)
 2. WHO (need or opportunity)
 3. THE (product name) is a (category)
 4. THAT (key benefit or reason to buy)
 5. UNLIKE (main competitor)
 6. OUR PRODUCT (primary difference or advantage)
3. Sources of product vision:

1. Domain experience — knowledge and expertise in a field.
 2. Product experience — finding ways to improve existing software.
 3. Customer experience — understanding feedback and user pain points.
 4. Prototyping and experimentation — testing ideas early.
-

5. Software Product Management (PM)

1. Product managers (PMs) are responsible for the product's vision, roadmap, and marketing direction.
 2. Key focus areas:
 1. Business needs — ensure the product supports company goals.
 2. Technology constraints — make development aware of technical limits or important issues.
 3. Customer experience — research and maintain contact with users to improve usability.
 3. Core responsibilities:
 1. Product vision management — maintain clarity and avoid “vision drift.”
 2. Product roadmap development — plan features and release schedule.
 3. User stories and scenarios — convert the vision into actionable tasks.
 4. Backlog creation and prioritization — organize development work effectively.
 5. Acceptance and customer testing — ensure the product meets its goals and user expectations.
 6. User interface evaluation — verify usability and avoid unnecessary complexity.
-

6. Product Prototyping

1. Prototyping builds an early version of the product to test ideas and demonstrate market potential.
 2. Benefits of prototyping:
 1. Gather user feedback and refine features.
 2. Test technology and feasibility before full development.
 3. Stages of prototyping:
 1. Feasibility prototype — proves the idea works and shows unique features.
 2. Customer demo prototype — adapts for user feedback and practical scenarios.
 4. After prototyping, rebuild the product for better security, reliability, and production quality.
-

7. Key Points Summary

1. Software products provide general functionality for broad markets.
2. Custom software is built for specific clients and often has a long life.
3. Execution models include stand-alone, hybrid, and SaaS.
4. Product vision defines purpose, users, and competitive advantage.
5. Product managers connect business, development, and customers.
6. Prototyping refines ideas and validates features before full production.

Software Disasters & Software Engineering Background

1. Cost & Impact of Software Disasters

1. Software errors cost the U.S. economy about **\$60 billion annually** due to rework, lost productivity, and damages.
 2. Faulty software can be **expensive, embarrassing, destructive, and deadly**.
 3. Poor software design or management can lead to large-scale economic loss, infrastructure failure, and even loss of life.
-

2. Major Software Disaster Cases

1. Mariner 1 Rocket Failure (1962)

- Cost: \$18.5M
- Cause: Programmer missed a single superscript bar in a formula → velocity data misread → rocket self-destructed.

2. Hartford Coliseum Roof Collapse (1978)

- Cost: \$70M + \$20M local economy damage
- Cause: CAD software design assumption error about steel support stress → roof failed under snow load.

3. CIA Gas Pipeline Explosion (1982)

- Cause: CIA inserted a bug in stolen pipeline control software → extreme pressure → massive explosion in Soviet gas system.

4. False Nuclear Launch Warning (1983)

- Cost: Potential global war avoided
- Cause: Bug in Soviet missile detection software misread sunlight reflections on clouds as missiles.

5. Therac-25 Radiation Overdose (1985)

- Cost: 3 dead, 3 critically injured
- Cause: Race condition allowed unsafe radiation levels due to software control failure.

6. Wall Street Black Monday (1987)

- Cost: \$500B in one day
- Cause: Automated computer trading systems triggered a cascade of sell orders, overwhelming the market.

7. AT&T Long-Distance Network Crash (1990)

- Cost: 75M calls missed, 200K airline reservations lost
- Cause: A single line of buggy code caused a cascade failure across switching centers.

8. Patriot Missile Failure (1991)

- Cost: 28 soldiers killed, 100 injured
- Cause: Rounding error in time calculation → missile defense failed to track incoming Scud.

9. Intel Pentium Floating Point Bug (1993)

- Cost: \$475M + brand credibility
- Cause: Flawed division lookup table → inaccurate floating-point division.

10. Ariane 5 Rocket Explosion (1996)

- Cost: \$500M
- Cause: 64-bit to 16-bit conversion overflow → guidance failure, rocket destroyed.

11. Mars Climate Orbiter Crash (1998)

- Cost: \$125M

- Cause: Software mismatch between imperial and metric units in thruster calculations.

12. British Passport System Failure (1999)

- Cost: £12.6M
- Cause: Poor testing and training; sudden surge in passport demand crashed the new system.

13. Y2K Bug (1999)

- Cost: ~\$500B global preparation
- Cause: Two-digit year storage misread "00" as 1900 → forced worldwide code fixes.

14. Dot-Com Bubble Collapse (2000)

- Cost: \$5T market value lost
- Cause: Overvalued startups ignored profitability; unrealistic growth assumptions.

15. ILOVEYOU Virus (2000)

- Cost: \$8.75B
- Cause: Worm spread via email and file sharing, deleting and damaging files.

16. Cancer Radiation Software (2000)

- Cost: 8 dead, 20 injured
- Cause: Software miscalculated dosage based on data entry order.

17. UK Child Support System Failure (2004)

- Cost: £539M and counting
- Cause: Overly complex IT system with thousands of bugs rolled out during agency restructuring.

18. FBI Virtual Case File Termination (2005)

- Cost: \$105M wasted
 - Cause: Mismanagement and outdated tech created an unusable case system.
-

3. Lessons & Importance of Software Engineering

1. Systematic Development Is Critical

- Ad hoc coding leads to errors with severe consequences.
- Large-scale failures show the need for disciplined engineering processes.

2. Testing and Validation Are Essential

- Many disasters caused by inadequate testing (Therac-25, AT&T crash, British Passports).

3. Proper Requirements and Design

- Misunderstood requirements or wrong assumptions (Hartford roof, Patriot missile) can be fatal.

4. Change Management

- Complex systems fail when updates aren't properly integrated or reviewed (AT&T, Y2K).

5. Communication & Documentation

- Clarity across teams avoids costly errors like metric-imperial mismatch (Mars Orbiter).
-

4. Background of Software Engineering

1. 1950s–1960s

- Birth of software engineering ideas; “software crisis” emerged.

- Old approach: “coding first, fix later” was slow and expensive.
- Waterfall model introduced but costly.

2. 1990s

- Demand for faster “time to market.”
- Concurrent engineering and productivity models introduced.

3. 2000s–Present

- Agile methodologies widely adopted to handle rapid change and complexity.

5. Four Key Elements of Software Engineering

1. **Methods** — techniques and approaches to software development.
2. **Tools** — support development and maintenance (IDEs, debuggers, version control).
3. **Procedures** — organized processes for design, coding, testing, and deployment.
4. **People** — developers, testers, managers, and users involved in the project.

6. Software Development Life Cycle (SDLC)

1. Requirements Analysis
2. Design
3. Implementation and Unit Testing
4. Integration
5. System Testing
6. Deployment and Maintenance

7. System Engineering Perspective

- A system is a purposeful collection of interrelated components to achieve objectives.
- Computer-based systems include hardware, software, data, people, and processes.
- Systems also follow a life cycle similar to SDLC but broader in scope.

8. Key Takeaways

1. Software engineering prevents costly, dangerous, and reputation-damaging disasters.
2. Rigorous design, testing, and project management are non-negotiable in critical systems.
3. Agile and modern methods exist to reduce time-to-market while controlling complexity and risk.
4. Human error combined with poor engineering discipline often leads to catastrophic failures.

Chapter 3: Agile Software Development

1. Rapid Software Development

1. Businesses need fast delivery because requirements constantly change.
 2. Plan-driven approaches can be too slow and rigid.
 3. Agile methods emerged in the late 1990s to shorten delivery time and adapt quickly.
-

2. Agile Development Core Ideas

1. Specification, design, implementation, and testing are **interleaved**, not strictly sequential.
 2. System built as **increments or versions**; stakeholders give feedback at each version.
 3. Frequent delivery of working software for evaluation.
 4. **Minimal documentation**, focus on working code.
 5. **Automated tools** (e.g., testing frameworks) support rapid development.
-

3. Agile vs Plan-Driven Development

Plan-Driven Development

- Pre-planned stages with defined outputs.
- Possible to be incremental but still heavily planned.
- Works for stable requirements and long-lifetime systems.

Agile Development

- Flexible and adaptive.

- Negotiation-driven outputs, built iteratively.
 - Suitable for fast-changing business needs and small/medium teams.
-

4. Agile Manifesto — Core Values

1. Individuals and interactions **over** processes and tools.
 2. Working software **over** comprehensive documentation.
 3. Customer collaboration **over** contract negotiation.
 4. Responding to change **over** following a plan.
-

5. Agile Principles

- Deliver working software frequently.
 - Welcome changing requirements.
 - Close cooperation with business stakeholders.
 - Build projects around motivated, skilled people.
 - Face-to-face communication is most effective.
 - Working software is the main progress measure.
 - Maintain a sustainable pace of development.
 - Continuous attention to technical excellence and simplicity.
-

6. Agile Method Applicability

1. Product development for small/medium products (e.g., apps, software tools).

2. Custom systems with committed customer involvement and minimal external regulations.
-

7. Extreme Programming (XP)

- Highly influential agile method, focused on **technical practices**.

Key XP Features

1. Frequent small releases (every 2 weeks or less).
2. Incremental builds, tests must pass before acceptance.
3. Full-time customer involvement in the team.
4. Pair programming & collective code ownership.
5. Constant refactoring to keep code simple and adaptable.

Important XP Practices

- **User stories** — requirements captured as short scenarios.
 - **Test-first development** — write tests before code (TDD).
 - **Refactoring** — continuous code improvement to make change easier.
 - **Automated testing** — regression checks after each change.
-

8. Test-First Development (TDD)

1. Tests clarify requirements before coding begins.
2. Tests written as executable programs (e.g., JUnit).
3. All tests run automatically whenever new code is added.

4. Benefits: immediate error detection, clear goals.
 5. Challenges: incomplete tests, difficult UI testing, uncertain coverage.
-

9. Pair Programming

1. Two programmers work together at one workstation.
 2. Provides informal peer review and spreads knowledge across the team.
 3. Encourages better design and refactoring.
 4. Often more effective than two individuals working separately.
-

10. Agile Project Management (Scrum)

Scrum Phases

1. **Initial planning** — define objectives, architecture, and backlog.
2. **Sprint cycles** — fixed 2–4 week iterations building product increments.
3. **Project closure** — finalize documentation, review lessons learned.

Scrum Roles

- **Scrum Master** — shields team from distractions, facilitates meetings.
- **Product Owner** — manages product backlog and priorities.
- **Team Members** — self-organize to complete sprint goals.

Scrum Process

- Daily stand-ups (short meetings) to share progress and problems.

- Sprint reviews to show completed work to stakeholders.
- Backlog updated regularly based on feedback.

Benefits of Scrum

- Manageable work chunks and faster visible progress.
- Improves communication and trust with customers.
- Handles unstable requirements well.

11. Scaling Agile Methods

1. Agile works best for **small, co-located teams**; scaling is challenging.
2. **Scaling up** — adapting agile for large systems needing multiple teams.
3. **Scaling out** — spreading agile across an organization.

Challenges in Large Projects

- Many teams and time zones reduce informal communication.
- Long system lifetimes require more documentation.
- Regulatory and contractual requirements can conflict with agile's flexibility.
- Difficult to maintain continuous integration across many components.

Multi-Team Solutions

- “Scrum of Scrums” — team representatives meet daily for coordination.
 - Product architects ensure coherent architecture.
 - Release alignment keeps system increments synchronized.
-

12. Agile & Plan-Driven Combination

- Many organizations use a **hybrid approach**:
 - Agile for flexible parts and customer feedback.
 - Plan-driven for high-risk, regulated, or long-lifetime components.
 - Balance depends on system size, regulatory needs, expected lifetime, and team skill.
-

13. Key Takeaways

1. Agile is about **rapid, flexible, and customer-focused** development.
2. XP introduced many core agile technical practices (TDD, refactoring, pair programming).
3. Scrum provides a widely used **management framework** for agile projects.
4. Large systems often need a mix of agile and plan-driven methods to handle complexity.
5. Good communication, skilled teams, and customer involvement are critical to agile success.